

AG2 - Actividad Guiada 2

Nombre: Raúl Sánchez Mondéjar

Link: <https://colab.research.google.com/drive/xxxxxxxxxxxxxxxxxxxxxxxx>

Github: https://github.com/rsmondejar/O3MIAR_10_B_2025-26_Algoritmos-de-Optimizacion_Actividad_Guiada_2

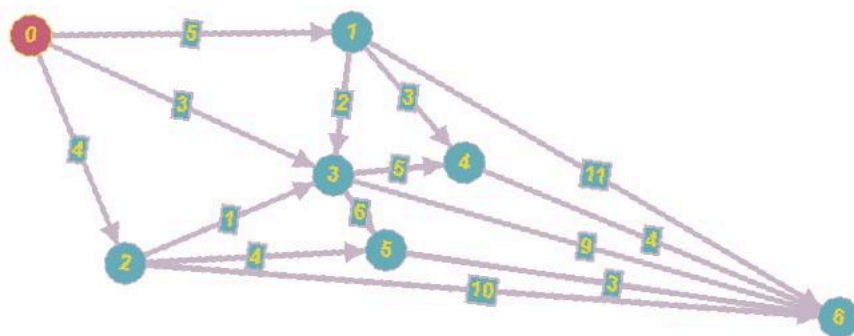
```
1 import math
```

Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: "en una secuencia optima de decisiones, toda sub-secuencia también es óptima" (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.



*Consideramos una tabla $TARIFAS(i,j)$ para almacenar todos los precios que nos ofrecen los embarcaderos.

*Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
1 # Viaje por el río - Programación dinámica
2 #####
3
4 TARIFAS = [
5     [0, 5, 4, 3, float("inf"), 999, 999], # desde nodo 0
6     [999, 0, 999, 2, 3, 999, 11], # desde nodo 1
7     [999, 999, 0, 1, 999, 4, 10], # desde nodo 2
8     [999, 999, 999, 0, 5, 6, 9],
9     [999, 999, 999, 999, 0, 999, 4],
10    [999, 999, 999, 999, 999, 0, 3],
11    [999, 999, 999, 999, 999, 999, 0]
12 ]
13
14 # 999 se puede sustituir por float("inf") del modulo math
15 TARIFAS
```

```
[[0, 5, 4, 3, inf, 999, 999],
 [999, 0, 999, 2, 3, 999, 11],
 [999, 999, 0, 1, 999, 4, 10],
 [999, 999, 999, 0, 5, 6, 9],
 [999, 999, 999, 999, 0, 999, 4],
 [999, 999, 999, 999, 999, 0, 3],
 [999, 999, 999, 999, 999, 999, 0]]
```

```
[999, 999, 999, 999, 999, 0, 3],
[999, 999, 999, 999, 999, 999, 0]]
```

```
1 # Calculo de la matriz de PRECIOS y RUTAS
2 # PRECIOS - contiene la matriz del mejor precio para ir de un nodo a otro
3 # RUTAS - contiene los nodos intermedios para ir de un nodo a otro
4 #####
5 def Precios(TARIFAS):
6     #####
7     # Total de Nodos
8     N = len(TARIFAS[0])
9
10    # Inicialización de la tabla de precios
11    PRECIOS = [[9999]*N for i in [9999]*N] # n x n
12    RUTA = [[""]*N for i in [""]*N]
13
14    # Se recorren todos los nodos con dos bucles(origen - destino)
15    # para ir construyendo la matriz de PRECIOS
16    for i in range(N-1):
17        for j in range(i+1, N):
18            MIN = TARIFAS[i][j]
19            RUTA[i][j] = i
20
21            for k in range(i, j):
22                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
23                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j])
24                    RUTA[i][j] = k
25                    PRECIOS[i][j] = MIN
26
27    return PRECIOS, RUTA
```

```
1 PRECIOS, RUTA = Precios(TARIFAS)
2 # print(PRECIOS[0][6])
3
4 print("PRECIOS")
5 for i in range(len(TARIFAS)):
6     print(PRECIOS[i])
7
8 print("\nRUTA")
9 for i in range(len(TARIFAS)):
10    print(RUTA[i])
```

```
PRECIOS
[9999, 5, 4, 3, 8, 8, 11]
[9999, 9999, 999, 2, 3, 8, 7]
[9999, 9999, 9999, 1, 6, 4, 7]
[9999, 9999, 9999, 9999, 5, 6, 9]
[9999, 9999, 9999, 9999, 9999, 999, 4]
[9999, 9999, 9999, 9999, 9999, 9999, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

RUTA
['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']
```

```
1 # Calculo de la ruta usando la matriz RUTA
2 def calcular_ruta(RUTA, desde, hasta):
3     if desde == RUTA[desde][hasta]:
4         # if desde == hasta:
5         # print("Ir a :" + str(desde))
6         return desde
7     else:
8         return str(calcular_ruta(RUTA, desde, RUTA[desde][hasta])) + ',' + str(RUTA[desde][hasta])
9
10
11 print("\nLa ruta es:")
12 calcular_ruta(RUTA, 0, 6)
```

```
La ruta es:
'0,2,5'
```

Haz doble clic (o pulsa Intro) para editar

▼ Problema de Asignacion de tarea

```

1 # Asignacion de tareas - Ramificación y Poda
2 #####
3 #   T A R E A
4 #   A
5 #   G
6 #   E
7 #   N
8 #   T
9 #   E
10
11 COSTES = [
12     [11, 12, 18, 40],
13     [14, 15, 13, 22],
14     [11, 17, 19, 23],
15     [17, 14, 20, 28]
16 ]

```

```

1 # Calculo del valor de una solucion parcial
2 def valor(S, COSTES):
3     VALOR = 0
4     for i in range(len(S)):
5         VALOR += COSTES[S[i]][i]
6     return VALOR
7
8
9 # Teniendo en cuenta que los indices de columnas y vectores comienzan por cero.
10 """En este ejemplo:
11 - el 3 sería la primera columna y la fila con índice 3, equivaldría al 17
12 - el 2 sería la segunda columna y la fila con índice 2, equivaldría al 17
13 """
14 print(valor((3, 2,), COSTES))
15
16 """En este ejemplo:
17 - el 0 sería la primera columna y la fila con índice 0, equivaldría al 11
18 - el 2 sería la segunda columna y la fila con índice 2, equivaldría al 17
19 """
20 print(valor((0, 2,), COSTES))
21
22 """En este ejemplo:
23 - el 3 sería la primera columna y la fila con índice 3, equivaldría al 17
24 - el 2 sería la segunda columna y la fila con índice 2, equivaldría al 17
25 - el 1 sería la tercera columna y la fila con índice 1, equivaldría al 13
26 - el 0 sería la cuarta columna y la fila con índice 0, equivaldría al 40
27 """
28 print(valor((3, 2, 1, 0), COSTES))

```

```

34
28
87

```

```

1 # Coste inferior para soluciones parciales
2 # (1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al agente 1
3
4 def CI(S, COSTES):
5     VALOR = 0
6     # Valores establecidos
7     for i in range(len(S)):
8         # Descomentar esta linea para testear
9         # print([i, S[i], COSTES[i][S[i]]])
10        VALOR += COSTES[i][S[i]]
11
12    # Estimacion
13    for i in range(len(S), len(COSTES)):
14        VALOR += min([COSTES[j][i] for j in range(len(S), len(COSTES))])
15    return VALOR
16
17
18 def CS(S, COSTES):
19     VALOR = 0
20     # Valores establecidos
21     for i in range(len(S)):
22        VALOR += COSTES[i][S[i]]
23
24    # Estimacion
25    for i in range(len(S), len(COSTES)):
26        # Descomentar esta linea para testear
27        # print([j, i, COSTES[j][i] for j in range(len(S), len(COSTES))])
28        VALOR += max([COSTES[j][i] for j in range(len(S), len(COSTES))])
29    return VALOR
30

```

```
31
32 CI((0, 1), COSTES)
```

68

```
1 # Genera tantos hijos como como posibilidades haya para la siguiente elemento de la tupla
2 # (0,) -> (0,1), (0,2), (0,3)
3 def crear_hijos(NODO, N):
4     HIJOS = []
5     for i in range(N):
6         if i not in NODO:
7             HIJOS.append({'s': NODO + (i,)})
8     return HIJOS
```

```
1 crear_hijos((0,) , 4)
```

```
[{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]
```

```
1 def ramificacion_y_poda(COSTES):
2     # Construccion iterativa de soluciones(arbol). En cada etapa asignamos un agente(ramas).
3     # Nodos del grafo { s:(1,2),CI:3,CS:5 }
4     # print(COSTES)
5     DIMENSION = len(COSTES)
6     MEJOR_SOLUCION = tuple(i for i in range(len(COSTES)))
7     CotaSup = valor(MEJOR_SOLUCION, COSTES)
8     # print("Cota Superior:", CotaSup)
9
10    NODOS = []
11    NODOS.append({'s': (), 'ci': CI((), COSTES)})
12
13    iteracion = 0
14
15    while (len(NODOS) > 0):
16        iteracion += 1
17
18        nodo_prometedor = [min(NODOS, key=lambda x: x['ci'])][0]['s']
19        # print("Nodo prometedor:", nodo_prometedor)
20
21        # Ramificacion
22        # Se generan los hijos
23        HIJOS = [
24            {'s': x['s'], 'ci': CI(x['s'], COSTES)}
25            for x in crear_hijos(nodo_prometedor, DIMENSION)
26        ]
27
28        # Revisamos la cota superior y nos quedamos con la mejor solucion si llegamos a una solucion final
29        NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION]
30        if len(NODO_FINAL) > 0:
31            # print("\n*****Soluciones:", [x for x in HIJOS if len(x['s']) == DIMENSION ] )
32            if NODO_FINAL[0]['ci'] < CotaSup:
33                CotaSup = NODO_FINAL[0]['ci']
34                MEJOR_SOLUCION = NODO_FINAL
35
36        # Poda
37        HIJOS = [x for x in HIJOS if x['ci'] < CotaSup]
38
39        # Añadimos los hijos
40        NODOS.extend(HIJOS)
41
42        # Eliminamos el nodo ramificado
43        NODOS = [x for x in NODOS if x['s'] != nodo_prometedor]
44
45    print(
46        "La solucion final es:", MEJOR_SOLUCION, " en ",
47        iteracion, " iteraciones", " para dimension: ", DIMENSION
48    )
49
50
51 ramificacion_y_poda(COSTES)
```

```
La solucion final es: [{'s': (1, 2, 0, 3), 'ci': 64}] en 10 iteraciones para dimension: 4
```

▼ Descenso del gradiente

```
1 import math # Funciones matematicas
2 import matplotlib.pyplot as plt # Generacion de gráficos (otra opcion seaborn)
3 import numpy as np # Tratamiento matriz N-dimensionales y otras (fundamental!)
4 # import scipy as sc
```

```
5
6 import random
```

Vamos a buscar el minimo de la funcion paraboloides :

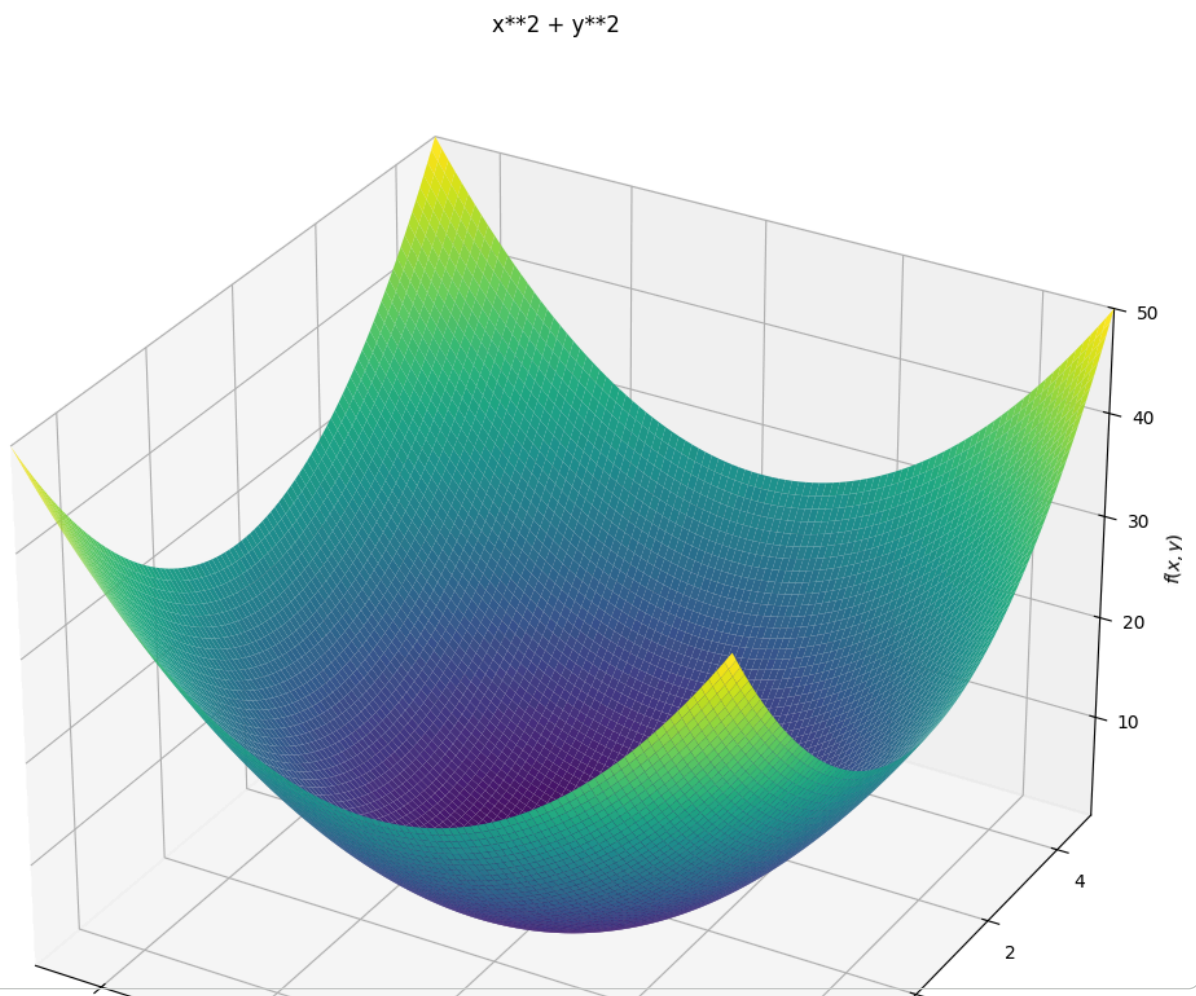
$$f(x) = x^2 + y^2$$

Obviamente se encuentra en $(x,y)=(0,0)$ pero probaremos como llegamos a él a través del descenso del gradiente.

```
1 # Definimos la funcion
2 # Paraboloides
3 f = lambda X: X[0]**2 + X[1]**2 #Funcion
4 df = lambda X: [ 2 * X[0] , 2 * X[1]] #Gradiente
5
6 df([1, 2])
```

[2, 4]

```
1 from sympy import symbols
2 from sympy.plotting import plot
3 from sympy.plotting import plot3d
4
5 x, y = symbols('x y')
6 plot3d(
7     x**2 + y**2,
8     (x, -5, 5),
9     (y, -5, 5),
10    title='x**2 + y**2',
11    size=(10, 10)
12 )
```

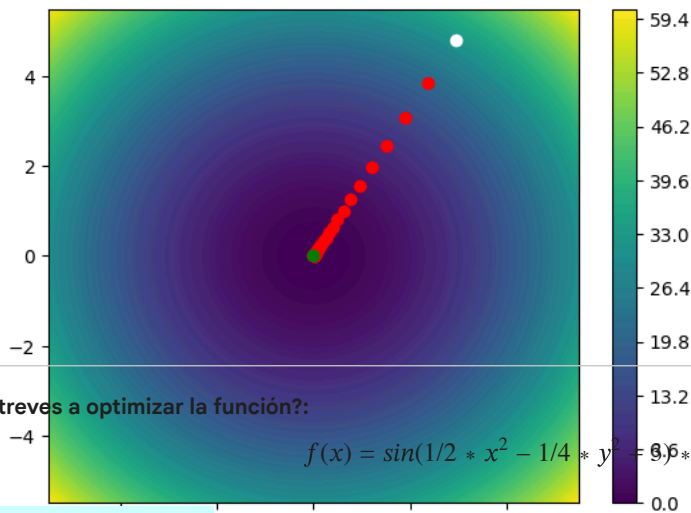


```
1 # Prepara los datos para dibujar mapa de niveles de Z
2 resolucion = 100
3 rango = 5.5
4
5 X = np.linspace(-rango, rango, resolucion)
6 Y = np.linspace(-rango, rango, resolucion)
7 Z = np.zeros((resolucion, resolucion))
8 for ix, x in enumerate(X):
9     for iy, y in enumerate(Y):
10        Z[iy, ix] = f([x, y])
```

```

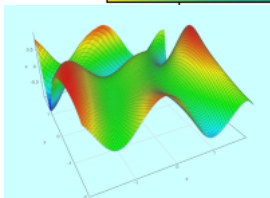
11
12 # Pinta el mapa de niveles de Z
13 plt.contourf(X, Y, Z, resolution)
14 plt.colorbar()
15
16 # Generamos un punto aleatorio inicial y pintamos de blanco
17 P = [random.uniform(-5, 5), random.uniform(-5, 5)]
18 plt.plot(P[0], P[1], "o", c="white")
19
20 # Tasa de aprendizaje (Learning Rate). Fija. Sería más efectivo reducirlo a medida que nos acercamos.
21 TA = .1
22
23 # Iteraciones:50
24 for _ in range(50):
25     grad = df(P)
26     # print(P,grad)
27     P[0], P[1] = P[0] - TA*grad[0], P[1] - TA*grad[1]
28     plt.plot(P[0], P[1], "o", c="red")
29
30 # Dibujamos el punto final y pintamos de verde
31 plt.plot(P[0], P[1], "o", c="green")
32 plt.show()
33 print("Solucion:", P, f(P))

```



¿Te atreves a optimizar la función?:

$$f(x) = \sin(1/2 * x^2 - 1/4 * y^2 + 3) * \cos(2 * x + 1 - e^y)$$



774026027e-05, 6.862666141485826e-05] 6.483744877931807e-09

```

1 #Definimos la funcion
2 f= lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3) *math.cos(2*X[0] + 1 - math.exp(X[1]))
3

```